

TECHNICAL REPORT

Industrial Rim Defect Classifier

Version 5 — Three-Backbone Ensemble with CBAM Attention

Target Performance

F1-Score ≥ 0.97 | AUC-ROC ≥ 0.97

Architecture Summary

EfficientNet-B2 1408 features	ResNet-50 2048 features	ConvNeXt-Tiny 768 features
---	-----------------------------------	--------------------------------------

64.3M Total Parameters | PyTorch + Albumentations | CUDA

1. Project Overview

This project builds an automated quality control system for **industrial rim inspection**. The goal is to classify wheel rim images as either **Good (label 0)** or **Faulty (label 1)** with very high reliability. The performance targets are F1-Score ≥ 0.97 and AUC-ROC ≥ 0.97 , which are demanding thresholds suited for real-world industrial deployment where misclassifying a faulty rim as good could cause safety issues.

The dataset contains **35,342 total images** split into 31,809 training samples and 3,533 validation samples. The training set has a class imbalance: 13,257 good rims vs. 18,552 faulty rims. This imbalance is addressed by multiple techniques described in Section 5.

Task	Binary image classification (Good / Faulty rim)
Dataset size	35,342 images (31,809 train, 3,533 val)
Class balance	13,257 Good 18,552 Faulty (before oversampling)
Image resolution	260 × 260 pixels (EfficientNet-B2 native size)
Target F1	≥ 0.97
Target AUC-ROC	≥ 0.97
Total parameters	64.3 million
Framework	PyTorch 2.x + Albumentations + torchvision

2. Preprocessing — Rim Crop Extraction

Before any image enters the model, a **preprocessing stage** isolates the rim region from the rest of the photo. Raw photos often contain background clutter such as floors, stands, or other rims that could confuse the model. Removing this context forces the network to focus exclusively on the rim surface, which is where all defects appear.

2.1 Hough Circle Transform

The rim localisation uses OpenCV's **Hough Circle Transform** (HoughCircles), which is a classical computer vision technique for detecting circular shapes in images. It works by analysing gradients in the image and voting in a 3D parameter space (centre x, centre y, radius).

The preprocessing pipeline runs as follows:

- Greyscale conversion:** Convert BGR to greyscale to reduce noise and speed up processing.
- Gaussian blur (kernel 9×9, $\sigma=2$):** Smooths out high-frequency noise that would create false gradient responses and cause the circle detector to find phantom circles.
- HoughCircles detection:** Searches for circles with radius between 100 and 600 pixels. The `dp=1.2` parameter sets the inverse ratio of the accumulator resolution to the image resolution, `minDist=200` prevents finding two circles too close to each other.

4. **Crop with margin:** Once the rim circle is found (centre cx, cy and radius r), the crop adds a margin of $(rim_width + 10) \times wide_scale$ pixels around the rim edge. This margin captures the full rim texture including the outermost surface.
5. **Fallback:** If no circle is detected, the full image is used. This is a safety net for unusual camera angles or heavily damaged rims.
6. **Resize to 260×260 and convert to RGB:** Final step normalises all images to the same size and colour space expected by the model.

Why 260×260?

EfficientNet-B2 was designed with a native input resolution of 260×260 pixels. Using this exact size means the compound scaling coefficients (depth, width, resolution) are in their optimal ratio, giving the best accuracy-to-VRAM trade-off. Using a smaller size would throw away spatial information; using a larger size would add computational cost without proportional accuracy gains.

3. Data Augmentation Strategy

Data augmentation synthetically expands the training set by applying random but realistic transformations to each image before it enters the model. This is one of the most important regularisation techniques in computer vision because it forces the model to be invariant to irrelevant variations (lighting, orientation, noise) while remaining sensitive to the actual defect features.

3.1 Why Albumentations Instead of torchvision

The training pipeline uses the **Albumentations** library instead of the standard torchvision transforms. The key reason is that Albumentations was built specifically for industrial and medical image tasks, offering transformations such as CLAHE and GaussNoise that are far more domain-relevant than generic image augmentations. It is also roughly 3–5× faster than torchvision for augmentation due to its NumPy/OpenCV backend.

3.2 Augmentation Pipeline (Training)

Each image goes through the following chain of transforms, applied randomly on every load:

Transform	Parameters	Reasoning
HorizontalFlip	$p=0.5$	Rims look the same mirrored; doubles training diversity for free
VerticalFlip	$p=0.5$	Rims are circular so vertical reflection is also valid
RandomRotate90	$p=0.5$	90° rotations are geometrically valid for round objects
CLAHE	$clip=3.0, grid=8 \times 8, p=0.5$	Contrast Limited Adaptive Histogram Equalisation enhances local contrast, making subtle

Transform	Parameters	Reasoning
RandomBrightnessContrast	limit= ± 0.3 , p=0.7	surface scratches and cracks visible that would otherwise blend into the background Simulates different factory lighting conditions; very high probability because lighting variation is the most common real-world variation
MotionBlur or GaussianBlur	blur=3-5, p=0.4	Simulates camera shake and focus variation in fast-moving inspection lines
GaussNoise	std=0.03-0.22, p=0.4	Adds sensor noise typical of industrial cameras operating under suboptimal conditions
Affine	shift= $\pm 5\%$, scale= $\pm 15\%$, rotate= $\pm 180^\circ$, p=0.6	Combined spatial distortion; the full 360° rotation range is appropriate because rims have no canonical 'upright' orientation
CoarseDropout	1-8 holes, 16-32px, p=0.3	Randomly erases rectangular patches, simulating occlusion and forcing the model to use the whole image rather than one salient region
Normalize	ImageNet mean/std	Shifts pixel values to the distribution the pretrained backbones expect
ToTensorV2	—	Converts NumPy HWC array to PyTorch CHW tensor

The validation transform applies only **Resize + Normalize + ToTensor** — no random augmentations. This is intentional: validation must be deterministic to give a fair, reproducible measure of model performance. If augmentation were applied at validation time, the score would vary between runs.

4. Model Architecture

The model is a **three-backbone ensemble** where each backbone independently extracts features from the input image and the resulting feature vectors are concatenated, passed through a channel-attention gate, and then through a classification head. This design is justified by a fundamental principle of ensemble learning: diverse models that make different errors will, when combined, be more accurate than any single model.

4.1 Why Three Backbones?

Each backbone has a different architectural inductive bias, meaning each has different implicit assumptions about how to process images:

- **EfficientNet-B2** — uses compound scaling (simultaneous depth + width + resolution scaling) and mobile-style inverted bottleneck blocks. It is highly efficient at capturing fine-grained local texture patterns, which is exactly where rim defects (scratches, cracks, corrosion) manifest.
- **ResNet-50** — uses deep residual connections with traditional large-kernel convolutions. It is strong at capturing global structural features and has the deepest proven track record in industrial inspection. Its 2048-dimensional feature vector is the richest of the three.
- **ConvNeXt-Tiny** — a 2022 architecture that modernises the ResNet design with large (7×7) depthwise convolutions, LayerNorm instead of BatchNorm, and GELU activations. Its depthwise-separable convolutions look at different spatial extents than standard convolutions, capturing texture patterns that EfficientNet and ResNet both miss.

Ensemble Diversity Principle

For an ensemble to outperform its individual members, the members must make different errors on different samples. EfficientNet, ResNet, and ConvNeXt were trained with different design philosophies and have never shared any architectural components, which maximises their error diversity. A three-model ensemble combining all three architectures will achieve a much lower joint error rate than any single backbone.

4.2 Feature Dimensions

EfficientNet-B2	1,408-dimensional feature vector after global average pooling
ResNet-50	2,048-dimensional feature vector after global average pooling
ConvNeXt-Tiny	768-dimensional feature vector after global average pooling
Concatenated	4,224-dimensional combined feature vector

4.3 CBAM — Convolutional Block Attention Module

Each backbone has a **CBAM block** inserted **before** its global pooling layer, operating directly on the 2D feature maps. CBAM applies two sequential attention mechanisms that teach the model to both select relevant feature channels and focus on relevant spatial regions.

Channel Attention

Channel attention answers the question: **"which feature maps matter?"** Each feature map at this stage corresponds to a learned detector (e.g., one map might detect edges, another might detect circular patterns). Channel attention compresses each feature map to a single number via both average-pooling and max-pooling, passes these through a small shared MLP, and generates a sigmoid gate between 0 and 1 for each channel. Feature maps with a high gate are amplified; those with a low gate are suppressed.

Spatial Attention

Spatial attention answers the question: **"where in the image should we look?"** It compresses the channel dimension to two summary maps (one via average-pooling across channels, one via max-pooling), concatenates them, and passes through a 7×7 convolution to produce a spatial weight map. This map highlights the rim surface and suppresses the background, which is especially useful because even after the Hough crop, some background may remain.

Why CBAM Instead of SE?

The previous version (v4) used a Squeeze-and-Excitation (SE) block, which provides channel attention only and operates on the already-pooled 1D feature vector. CBAM is superior here because it applies spatial attention on the **2D feature maps before pooling**, meaning the model can learn to ignore background regions before they contaminate the pooled features. SE blocks have no spatial awareness at all.

4.4 Fusion Attention Gate

After the three backbones produce their feature vectors and they are concatenated into a 4,224-dimensional vector, a second attention mechanism — the **fusion attention gate** — is applied. This is a small MLP (4224 → 264 → 4224) with a sigmoid output that produces a gate value for each of the 4,224 dimensions. The gate suppresses feature dimensions that are noisy or inconsistent across backbones and amplifies the ones that all three agree on.

4.5 Classification Head

The gated 4,224-dimensional vector is passed through a two-layer MLP:

- `Linear(4224 → 512)` + BatchNorm1d + SiLU activation + Dropout(0.45)
- `Linear(512 → 128)` + BatchNorm1d + SiLU activation + Dropout(0.30)
- `Linear(128 → 1)` — single logit, passed through sigmoid for probability

Why SiLU (Swish) instead of ReLU? SiLU ($x \times \text{sigmoid}(x)$) is smooth and non-monotonic, which has been shown to improve gradient flow in deep networks. It avoids the “dying ReLU” problem where neurons with negative input permanently output zero.

Why high dropout (0.45)? The input to the head is 4,224 features extracted from 64.3M parameters. Without strong regularisation, the head will memorise training samples rather than learn generalisable patterns. The 0.45 dropout on the first layer is deliberately aggressive because the feature space is very high-dimensional.

5. Training Strategy

5.1 Two-Phase Training

Training uses a **two-phase curriculum** that mirrors the standard transfer learning practice.

Phase 1: Backbone Frozen (Epochs 1–12)

All three backbone weights are frozen. Only the CBAM blocks, fusion attention gate, and classifier head are trained. The learning rate is set to $1r \times 10 = 1e-3$. This phase serves two purposes: first, it lets the classification head learn a good initial mapping from pretrained features to rim defects without corrupting the pretrained backbone weights before the head has stabilised. Second, it is much faster because gradient computation is skipped for most of the 64.3M parameters.

Phase 2: Full Fine-Tuning with Differential Learning Rates (Epochs 13–60)

All backbone weights are unfrozen. Each backbone gets a **different learning rate** to reflect the different amounts of fine-tuning each needs:

Parameter group	Learning rate	Reasoning
EfficientNet-B2	$1r \times 0.05 = 5e-6$	Well-pretrained; too much update would destroy ImageNet features
ResNet-50	$1r \times 0.05 = 5e-6$	Same; proven stable architecture, no need for large updates
ConvNeXt-Tiny	$1r \times 0.03 = 3e-6$	Most modern pretraining, even smaller LR to preserve it
Fusion attention + head	$1r = 1e-4$	Randomly initialised, needs full learning rate

5.2 Learning Rate Scheduling

Phase 1: OneCycleLR

The OneCycleLR scheduler starts at a low learning rate, ramps up to max_lr over the first 30% of warmup epochs, then anneals down with cosine decay. This “warm-up” period prevents the randomly initialised head from making large destructive gradient updates in the very first batches.

Phase 2: CosineAnnealingWarmRestarts (SGDR)

The cosine annealing scheduler periodically resets the learning rate to its initial value. Each restart cycle has length $T_0 = (\text{epochs} - \text{warmup}) / 3$. The rationale is that after annealing into a local minimum, a fresh restart allows the optimiser to escape that minimum and find a flatter, better-generalising one. Research has shown that the model at the end of each annealing cycle (when LR is near zero) tends to have better generalisation.

5.3 Focal Loss with Label Smoothing

Focal Loss

Instead of standard Binary Cross-Entropy, the pipeline uses **Focal Loss** with $\gamma=2.5$. Focal loss multiplies the standard BCE loss by a factor of $(1 - p_t)^\gamma$, where p_t is the model's predicted probability for the correct class. This means:

- **Easy examples** (correctly classified with high confidence, $p_t \approx 1.0$): the factor $(1 - 1.0)^{2.5} \approx 0$, so these contribute almost nothing to the gradient.
- **Hard examples** (misclassified or uncertain, $p_t \approx 0.5$): the factor $(1 - 0.5)^{2.5} \approx 0.18$, so these contribute much more to the gradient.

The result is that the model spends its parameter updates on the samples it finds difficult — precisely the borderline defects that matter most for industrial inspection. The gamma value of 2.5 is aggressive (the standard is 2.0), which makes sense given the large dataset size and the high performance target.

Label Smoothing (eps=0.05)

Label smoothing prevents the model from being overconfident by replacing hard targets (0.0 and 1.0) with softened versions (0.025 and 0.975). This is computed as: `target_smooth = target * 0.95 + 0.025`. When a model predicts exactly the hard target, it still experiences a small loss signal, which stops it from driving its logits to infinity and improves calibration on out-of-distribution samples.

Class-Weighted pos_weight

The focal loss is also given a `pos_weight` equal to `n_good / n_faulty`. This directly multiplies the loss for faulty samples, compensating for any residual class imbalance after oversampling. It is a standard technique from scikit-learn's `class_weight` parameter applied to PyTorch's `BCEWithLogitsLoss`.

5.4 Mixup and CutMix

On every training batch, the pipeline randomly applies either **Mixup** or **CutMix** with equal probability. Both create virtual training examples by combining pairs of images and their labels.

Mixup

Mixup creates a convex combination: `x_mixed =  $\lambda$  × x_i + (1- $\lambda$ ) × x_j` where λ is sampled from Beta(0.3, 0.3). The label is also a soft combination: `loss =  $\lambda$  × loss(x_i) + (1- $\lambda$ ) × loss(x_j)`. This creates training samples that sit in the continuous space between two real examples, preventing the model from drawing sharp decision boundaries and improving calibration.

CutMix

CutMix pastes a rectangular region from one image onto another. The patch size is determined by λ : `cut_size = image_size × sqrt(1- $\lambda$ )`. The label weight is adjusted to match the actual pixel proportion from each image. CutMix acts as a stronger spatial augmentation that forces the model to look at the entire image rather than fixating on one discriminative region, which is important for rim inspection where defects can appear anywhere on the rim.

5.5 Class Imbalance Handling

Three separate techniques are combined to address class imbalance:

7. **Random oversampling:** The minority class is duplicated with replacement until both classes are equal in size, changing the distribution from 13,257/18,552 to 18,552/18,552.
8. **WeightedRandomSampler:** The DataLoader uses a weighted sampler that gives each sample a selection probability inversely proportional to its class frequency. Even after oversampling, this provides an additional guarantee that each batch sees a balanced mix.
9. **Focal loss + pos_weight:** At the loss level, faulty examples are up-weighted again. This triple approach ensures the model never develops a systematic preference for predicting the majority class.

5.6 Exponential Moving Average (EMA)

An EMA copy of the model weights is maintained throughout training using $\text{decay}=0.9995$. After each optimiser step: `ema_weights = 0.9995 * ema_weights + 0.0005 * current_weights`. The EMA model is used for validation and checkpointing. This acts as a free ensemble over the last ~2000 update steps, producing a smoother weight trajectory that is less affected by the noise in individual mini-batch gradients.

Before saving the EMA model as a checkpoint, **BatchNorm recalibration** is performed: running mean and variance statistics in all BatchNorm layers are reset and re-accumulated using 50 training batches. This is necessary because the EMA weights are a blend of many past parameter states and the BN statistics were computed for the current (non-EMA) model.

5.7 Stochastic Weight Averaging (SWA)

Starting at epoch 40, a **Stochastic Weight Averaging** model is maintained alongside the EMA model. SWA averages the model weights at the end of each epoch after epoch 40. Research has shown that SWA finds flatter minima in the loss landscape, and flatter minima generalise better than sharp ones (Izmailov et al., 2018). The SWA model is saved separately as `model_v5_swa.pt` at the end of training and its validation F1 is compared against the best EMA checkpoint.

5.8 Gradient Accumulation

The effective batch size is `batch_size * accum_steps = 16 * 2 = 32`. Gradients are computed for 2 mini-batches before performing an optimiser step. This simulates training with a batch size of 32 while only holding 16 images in GPU memory at once. Larger effective batch sizes lead to more stable gradient estimates, which is especially important for the three-backbone architecture where the gradient signal must propagate through 64.3M parameters.

5.9 Mixed Precision Training (AMP)

All forward passes use **torch.amp.autocast**, which automatically runs eligible operations in float16 rather than float32. A GradScaler compensates for the reduced numerical range by scaling the loss before backward and unscaling before the optimiser step. This reduces VRAM usage by ~40% and speeds up convolution operations by 1.5–2× on NVIDIA GPUs with Tensor Cores.

5.10 Gradient Clipping

Gradients are clipped to a maximum L2-norm of 1.0 before each optimiser step. This prevents “gradient explosions” where one bad batch sends the parameters to a very different region of weight space. With three backbones and a complex attention mechanism, the gradient norm can occasionally spike, and clipping provides a hard upper bound on the size of any single update.

5.11 Early Stopping

Training stops early if the best validation F1 has not improved for 12 consecutive epochs. This prevents overfitting and saves compute time. The patience of 12 epochs is generous enough to allow the cosine annealing restarts to have their intended effect of escaping local minima.

6. Evaluation Methodology

6.1 Why F1-Score as the Primary Metric

Accuracy would be a misleading metric here because of the class imbalance. A model that predicts “Faulty” for every image would get 52.5% accuracy while being completely useless. F1-score is the harmonic mean of Precision and Recall:

$$F1 = \frac{2 \times TP}{2 \times TP + FP + FN}$$

It penalises equally for False Positives (good rims rejected as faulty, wasting production) and False Negatives (faulty rims approved, causing safety risk). AUC-ROC measures the model’s ability to rank faulty images above good ones across all possible thresholds and is threshold-independent.

6.2 Threshold Sweep

Rather than always using the default threshold of 0.5, the pipeline sweeps thresholds from 0.05 to 0.95 in steps of 0.05 and reports the best F1 at each. This is critical in practice because the optimal threshold depends on the cost asymmetry between false positives and false negatives in the deployment environment. The best threshold found during training is saved in the checkpoint and should be passed to the inference script.

6.3 Test-Time Augmentation (TTA)

During validation (by default every 5 epochs or when `--use-tta` is set), the model produces 5 predictions per image using different augmented views: original, horizontal flip, vertical flip, 90° rotation, and 270° rotation. The 5 sets of logits are averaged before sigmoid is applied. This free ensemble over views typically improves F1 by 0.5–1.5 percentage points.

7. Performance Optimisations

Several engineering optimisations were applied to reduce training time without affecting model accuracy. These are important given the dataset size (37k images) and model size (64.3M parameters).

Optimisation	Mechanism	Expected Speedup
cudnn.benchmark=True	cuDNN auto-selects fastest convolution kernel for the fixed input size	10–15% faster convolutions
TF32 precision	Tensor Float 32: 10-bit mantissa matmuls on Ampere+ GPUs	Up to 2× on RTX 30xx/40xx
torch.compile	Fuses multiple ops into single optimised GPU kernels (Triton backend)	20–40% per epoch after first
non_blocking transfers	CPU→GPU copy overlaps with GPU compute using pinned memory	Removes data transfer idle time
zero_grad(set_to_none=True)	Sets gradient buffers to None instead of zeroing; avoids a full memory write	Small but free speedup
prefetch_factor=4, workers=8	Each DataLoader worker pre-loads 4 batches while GPU is busy	Keeps GPU fully utilised
2× val/test batch size	No gradient storage needed at eval; can use larger batch	~50% faster validation

Gradient Checkpointing (Optional)

If VRAM is a bottleneck, run with `--grad-checkpoint`. This trades compute for memory by not storing intermediate activations during the forward pass; instead they are recomputed during the backward pass. This saves ~40% VRAM at the cost of ~25% more compute time per epoch. It is particularly useful when scaling to larger batch sizes or image sizes.

8. Bugs Fixed During Development

The following bugs were identified and fixed during the development process. Each represents a real failure mode encountered when running the script.

Bug 1: A.RandomErasing Does Not Exist

Error: AttributeError: module 'albumentations' has no attribute 'RandomErasing'

The Albumentations library never implemented RandomErasing (a torchvision concept). The correct Albumentations equivalent is `CoarseDropout`, which achieves the same effect by randomly erasing rectangular patches. The parameters were also updated to the new API: `num_holes_range`, `hole_height_range`, `hole_width_range` instead of the deprecated `max_holes`, `max_height`, `max_width`.

Bug 2: PIL Image Passed to Albumentations Transform

Error: TypeError on transform call — albumentations receives a PIL Image instead of a numpy array. Albumentations expects a NumPy array with shape (H, W, C) and is called as `transform(image=arr) ["image"]`. Torchvision expects a PIL Image and is called as `transform(img)`. The dataset's `__getitem__` was calling the transform directly on the PIL Image without converting it. Fixed by adding a branch that converts to numpy when albumentations is active.

Bug 3: Val Transform Always Used torchvision

Error: Runtime crash during validation — val transform is torchvision but `__getitem__` calls it as albumentations

The validation transform function always returned a `T.Compose` object, but after Bug 2 was fixed, `__getitem__` was now calling all transforms (including val) as albumentations-style. Fixed by making `make_val_transform` also return an `A.Compose` when albumentations is available.

Bug 4: Missing val_f1 in Validation Fallback Branch

Error: NameError: name 'val_f1' is not defined — on epochs where the full `_validate()` is skipped

The else branch that runs on non-TTA epochs only assigned `val_acc`, `val_prec`, `val_rec`, `val_auc`, missing `val_f1`. On the next epoch where `val_f1` was referenced in the print statement, Python raised a NameError. Fixed by adding `val_f1 = 0.0` to the fallback assignment.

Bug 5: ConvNeXt LayerNorm Crashes with 2D Input

Error: RuntimeError: permute(sparse_coo): number of dimensions in the tensor input does not match len(dims) i.e. input.dim()=2 is not equal to len(dims)=4

This was the most subtle bug. `base.classifier[0]` appears to be a plain `nn.LayerNorm` but is actually torchvision's `LayerNorm2d` wrapper, which internally calls `x.permute(0,2,3,1)` before normalising, expecting a 4D tensor (B, C, H, W). The model pools to (B, 768) before calling it, so the permute fails because 2D tensors cannot be permuted with 4 dimensions. Fixed by replacing `base.classifier[0]` with `nn.LayerNorm(768)`, which is a standard LayerNorm that works on any shape ending in 768.

Bug 6: Deprecated Albumentations API

Warning/Error: UserWarning: Argument(s) 'var_limit' are not valid for transform GaussNoise / ShiftScaleRotate is a special case of Affine transform

Two albumentations API changes in version 1.4+: (1) `GaussNoise(var_limit=...)` was replaced by `GaussNoise(std_range=...)` because std is more intuitive than variance. Values were converted: var

10–50 → std 0.03–0.22. (2) `ShiftScaleRotate` was deprecated in favour of the more general `A.Affine` which accepts the same parameters in a unified interface.

9. Full Hyperparameter Reference

<code>--epochs</code>	60 — Total training epochs
<code>--warmup-epochs</code>	12 — Epochs of backbone-frozen Phase 1
<code>--batch</code>	16 — Mini-batch size (effective 32 with accum-steps=2)
<code>--lr</code>	1e-4 — Base learning rate for head (backbone LRs derived from this)
<code>--img-size</code>	260 — EfficientNet-B2 native resolution
<code>--ema-decay</code>	0.9995 — EMA smoothing factor (~2000 effective update steps)
<code>--label-smooth</code>	0.05 — Label smoothing epsilon
<code>--grad-clip</code>	1.0 — Max gradient L2-norm
<code>--early-stop</code>	12 — Patience in epochs
<code>--mixup-alpha</code>	0.3 — Beta distribution parameter for Mixup/CutMix
<code>--focal-gamma</code>	2.5 — Focal loss hardness concentration
<code>--swa-start</code>	40 — Epoch at which SWA averaging begins
<code>--accum-steps</code>	2 — Gradient accumulation steps
<code>--val-split</code>	0.10 — 10% of data held out for validation
<code>--tta-freq</code>	5 — Run TTA validation every 5 epochs
<code>--workers</code>	8 — DataLoader worker processes
<code>--prefetch-factor</code>	4 — Batches pre-loaded per worker
<code>--grad-checkpoint</code>	Off by default — enable to reduce VRAM by ~40%
<code>--compile</code>	Off by default — enable for 20-40% faster epochs on PyTorch 2.0+

10. How to Run

Standard Training

```
python train_cnn_v2.py
```

With Speed Optimisations (Recommended)

```
python train_cnn_v2.py --compile --workers 8 --prefetch-factor 4
```

If VRAM is Limited

```
python train_cnn_v2.py --grad-checkpoint --accum-steps 4 --batch 8
```

Resume from Checkpoint

```
python train_cnn_v2.py --resume assets/model_v5.pt
```

Output Files

- `assets/model_v5.pt` — Best EMA checkpoint (by validation F1)
- `assets/model_v5_swa.pt` — SWA model after epoch 40
- `assets/val_split.csv` — Validation split for reproducibility
- `runs/rim_v5/` — TensorBoard logs (loss, F1, precision, recall, AUC)